

Design of an On-Chip Trainable GRU-based Channel Equalizer On FPGA

Roshan Sharma (2204229), Final Year B.Tech Student
Project Supervisor: Dr. Nandakumar Nambath

Abstract—This project presents a Gated Recurrent Unit (GRU) based hardware accelerator with on-chip training using Back Propagation Through Time (BPTT) for adaptive channel equalization in communication systems, particularly suited for scenarios where the channel characteristics vary over time. It utilizes IEEE 754 single precision floating point arithmetic for high precision inference and faster gradient descent. Hardware was serialized to offset the resource constraint bottleneck. The design was verified through RTL simulation and implementation in DE-10 Lite, and successfully performed inference ($MSE < 0.2\%$) in hardware and simulation results and the error successfully converged when training on FPGA in a small sample.

Index Terms—FPGA, Verilog, GRU, On-Chip Training, BPTT, FPU, Channel Equalization, RNN, Hardware Accelerator, QPSK.

I. INTRODUCTION

Neural networks have become widely popular due to their simpler architecture and adaptive nature. One such application is in communication systems where, due to rise in data traffic, has shifted to higher order modulation techniques and symbol rates [2]. However, it introduces severe non-linear impairments which traditional Digital Signal Processing (DSP) techniques require higher complexity to compensate for.

Traditionally used neural network-based equalizers like Convolutional Neural Networks (CNN) or Multilayer Perceptrons (MLP) [1] are feed-forward architectures and cannot effectively compensate for the Inter-Symbol Interference (ISI) which creates temporal dependencies between the inputs as they are memoryless.

To mitigate this issue, Recurrent Neural Networks (RNN), specifically Gated Recurrent Units (GRU) can be used as their internal hidden state acts as a memory, allowing them to implicitly model the ISI.

Current hardware implementations of NN-based equalizers typically use offline training, where the model is trained on a computer using a static dataset and then the weights are exported onto the chip. This approach, although simpler and resource-efficient, cannot accommodate for the changing channel conditions [6].

This project proposes a modular and training capable design for adaptive channel conditions. By implementing Back Propagation Through Time (BPTT) algorithm in Verilog, the FPGA can calculate error gradients and perform gradient descent to update the weights.

II. OBJECTIVES

A. Selection of RNN type

As vanilla recurrent neural networks (RNN) suffers from exploding and vanishing gradient problem, advanced models such as Long-Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) overcomes these issues through gating mechanisms, by selectively forgetting or remembering information, thereby these gates create direct paths in the network.

B. Design of Python Model

The next goal was to design the whole architecture in Python using the PyTorch library and simulating it to observe if the weights are settling and mean-squared error (MSE) is converging and then exporting the weights for hardware simulation. This also involved debugging where the hardware modules are failing while comparing with the Python model.

C. Design of Verilog Inference Modules

This stage involved building the forward pass (inference) modules in Verilog and testing it on RTL(Register Transfer Level) simulation and FPGA using the exported weights from Python. This stage also involved choosing the type of implementation of activation functions, i.e., the Tanh and Sigmoid functions which could be implemented in hardware using Look-up tables (LUTs), Piecewise linear approximation (PWL) or Taylor series expansion [2].

D. Design of Training Modules

The final stage involved building the backpropagation through time (BPTT) modules and modifying the inference modules to store intermediate gate testing it on Behavioral simulation and FPGA. This required learning to derive the necessary equations to be implemented and how to optimize it for hardware implementation.

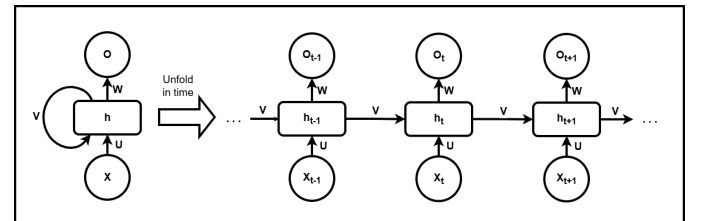


Fig. 1. Basic RNN Working

III. THEORY

The proposed design forms on basis of the following theory:

A. RNN Architecture

A standard RNN processes sequential data by maintaining a “hidden state” vector (h_t) that acts as a short-term memory [2]. At every time step t , this state is updated by combining the current input (x_t) and the previous state (h_{t-1}) using a tanh activation function:

$$h_t = \tanh(W_{zh}x_t + W_{hh}h_{t-1} + b) \quad (1)$$

And the output is given by:

$$o = W_{fc}h_t + b_{fc} \quad (2)$$

Where W and b refer to the corresponding weights and biases, respectively.

As depicted in Fig. 1, the basic cell unfolds in time for multiple numbers of times, which is equal to the input sequence length. This recurrent loop of hidden state allows the network to pass information forward through time, enabling it to model temporal dependencies.

But training an RNN requires Backpropagation Through Time (BPTT), where error gradients are propagated backward from the last time step to the first [3]. This mathematically involves repeated multiplication of the recurrent weight matrix (W_{hh}).

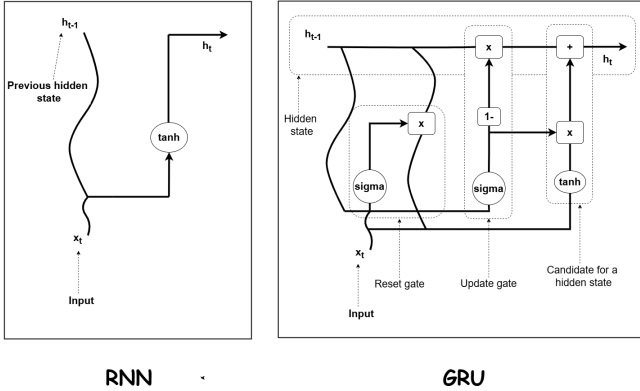


Fig. 2. Basic RNN vs GRU structure

Vanishing Gradient: If the weights are small (< 1), the gradients shrink exponentially as they travel back. The network effectively “stops learning” from early inputs, making it unable to capture long-term dependencies.

Exploding Gradient: If weights are large (> 1), gradients grow exponentially, causing numerical overflow (NaNs) and destabilizing the hardware training loop.

To resolve this, this project implements the Gated Recurrent Unit (GRU) [5] as shown in Fig. 2. Unlike a simple RNN, the GRU introduces explicit gating mechanisms, the Update Gate (z_t) and Reset Gate (r_t), to regulate information flow.

The equations implemented in the Verilog hardware are:

- 1) **Reset gate** (r_t): Decides how much past information to discard.

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (3)$$

- 2) **Update gate** (z_t): Decides how much past information to keep.

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (4)$$

- 3) **Candidate state** (n_t): Computes new memory content.

$$n_t = \tanh(W_h x_t + r_t \odot (U_h h_{t-1}) + b_h) \quad (5)$$

- 4) **Final hidden state** (h_t): Mixing old and new memory.

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1} \quad (6)$$

The final equation creates an additive path for gradients. If $z_t \approx 1$, the previous state is copied directly to the current state. This allows gradients to flow backward unchanged over many time steps, effectively solving the vanishing and exploding gradient problem and enabling the hardware to learn long sequences stably.

The output equation remains the same:

$$o = W_{fc}h_t + b_{fc} \quad (7)$$

B. Backpropagation Through Time (BPTT)

Training a Gated Recurrent Unit (GRU) differs from standard feedforward networks because the parameters (W, U, b) are shared across all time steps. To compute gradients, the network is mathematically “unfolded” through time, creating a deep feed-forward network with T layers (where T is the sequence length). The training process, known as Backpropagation Through Time (BPTT), involves propagating error signals from the last time ($t = T$) step back to the first ($t = 1$) [3].

The objective of training is to minimize the Loss (L), which is taken to be the mean squared error (MSE) between the target and the predicted values as shown in Equation 8. BPTT calculates how much each weight contributed to this error by computing the gradient of the loss with respect to the weights (ΔW). These gradients are then subtracted from the previous weights’ value with a scaled parameter known as the learning rate (η). The inference is again run with the updated weights and then the loss is again calculated. This process is done again and again and eventually the weights settle and therefore the model is trained. This process is called Stochastic Gradient Descent (SGD).

$$L(\text{Loss}) = \frac{1}{2} \|y_D - y_P\|^2 \quad (8)$$

Where,

$$y_D = \begin{pmatrix} y_{I,D} \\ y_{Q,D} \end{pmatrix} \quad (9)$$

(Refers to the target QPSK symbol comprising of In-phase and Quadrature components.)

and

$$y_P = \begin{pmatrix} y_{I,P} \\ y_{Q,P} \end{pmatrix} \quad (10)$$

(Refers to the predicted QPSK symbol comprising of In-phase and Quadrature components.)

The gradient of Loss with respect to predicted output,

$$\frac{\partial \mathcal{L}}{\partial y_P} = (y_D - y_P) \quad (11)$$

Output layer gradients:

$$\frac{\partial \mathcal{L}}{\partial W_{(fc)}} = \left(\frac{\partial \mathcal{L}}{\partial y_P} \right) h_T^T \quad (12)$$

$$\frac{\partial \mathcal{L}}{\partial h_T} = W_{(fc)}^T \left(\frac{\partial \mathcal{L}}{\partial y_P} \right) \quad (13)$$

Gate gradients (per timestep):

- Candidate:

$$\frac{\partial \mathcal{L}}{\partial n_t} = \left(\frac{\partial \mathcal{L}}{\partial h_t} \right) \odot (1 - z_t) \quad (14)$$

$$\frac{\partial \mathcal{L}}{\partial a_n^t} = \left(\frac{\partial \mathcal{L}}{\partial n_t} \right) \odot (1 - n_t^2) \quad (15)$$

- Update gate:

$$\frac{\partial \mathcal{L}}{\partial z_t} = \left(\frac{\partial \mathcal{L}}{\partial h_t} \right) \odot (h_{(t-1)} - n_t) \quad (16)$$

$$\frac{\partial \mathcal{L}}{\partial a_z^t} = \left(\frac{\partial \mathcal{L}}{\partial z_t} \right) \odot z_t(1 - z_t) \quad (17)$$

- Reset gate:

$$\frac{\partial \mathcal{L}}{\partial r_t} = \left(U_{(n)}^T \left(\frac{\partial \mathcal{L}}{\partial a_n^t} \right) \right) \odot h_{(t-1)} \quad (18)$$

$$\frac{\partial \mathcal{L}}{\partial a_r^t} = \left(\frac{\partial \mathcal{L}}{\partial r_t} \right) \odot r_t(1 - r_t) \quad (19)$$

Weight gradients:

- Candidate:

$$\frac{\partial \mathcal{L}}{\partial W_{(n)}} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_n^t} \right) x_t^T \quad (20)$$

$$\frac{\partial \mathcal{L}}{\partial U_{(n)}} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_n^t} \right) (r_t \odot h_{(t-1)})^T \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial b_n} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_n^t} \right) \quad (22)$$

- Update gate:

$$\frac{\partial \mathcal{L}}{\partial W_{(z)}} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_z^t} \right) x_t^T \quad (23)$$

$$\frac{\partial \mathcal{L}}{\partial U_{(z)}} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_z^t} \right) h_{(t-1)}^T \quad (24)$$

$$\frac{\partial \mathcal{L}}{\partial b_z} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_z^t} \right) \quad (25)$$

- Reset gate:

$$\frac{\partial \mathcal{L}}{\partial W_{(r)}} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_r^t} \right) x_t^T \quad (26)$$

$$\frac{\partial \mathcal{L}}{\partial U_{(r)}} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_r^t} \right) h_{(t-1)}^T \quad (27)$$

$$\frac{\partial \mathcal{L}}{\partial b_r} = \sum \left(\frac{\partial \mathcal{L}}{\partial a_r^t} \right) \quad (28)$$

Gradient Propagation through timesteps:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial h_{(t-1)}} = & \left(\frac{\partial \mathcal{L}}{\partial h_t} \right) \odot z_t + \left(U_{(n)}^T \left(\frac{\partial \mathcal{L}}{\partial a_n^t} \right) \right) \odot r_t \\ & + U_{(z)}^T \left(\frac{\partial \mathcal{L}}{\partial a_z^t} \right) + U_{(r)}^T \left(\frac{\partial \mathcal{L}}{\partial a_r^t} \right) \end{aligned} \quad (29)$$

Weight update:

$$\theta = \theta - \eta \left(\frac{\partial \mathcal{L}}{\partial \theta} \right) \quad (30)$$

where η is the learning rate, θ represents any weight parameter and the summation for weight gradients is from $t = 1$ to $t = T$.

Also, a_r^t , a_z^t and a_n^t are given by:

$$a_r^t = W_{(r)}x_t + U_{(r)}h_{(t-1)} + b_r \quad (31)$$

$$a_z^t = W_{(z)}x_t + U_{(z)}h_{(t-1)} + b_z \quad (32)$$

$$a_n^t = W_{(n)}x_t + U_{(n)}(r_t \odot h_{(t-1)}) + b_n \quad (33)$$

C. Activation Functions

Activation functions are integral part of neural networks. They help the network learn complex non-linearities by transforming the linear sum of inputs into non-linear output. Without activations functions, the effective structure of the network would become just a linear transformation of inputs, which would not learn any complex patterns present in the input.

In GRUs, two activation functions are used, Sigmoid and Tanh. Sigmoid is used to generate the reset and update gates, whereas tanh is used to generate the Candidate state as shown in Equations 3, 4 and 5.

Tanh is bounded between -1 to 1 whereas Sigmoid is bounded between 0 to 1 as shown in Fig. 3.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (34)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (35)$$

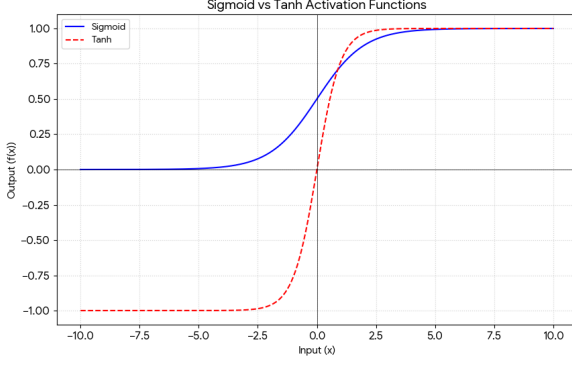


Fig. 3. Sigmoid and tanh activation functions used in the GRU architecture.

IV. METHODOLOGY

A. Model Configuration and Specifications

Before mapping the algorithm to hardware, the specific hyperparameters of the GRU network were defined to target the channel equalization task. To validate the hardware design, a “Golden Model” was first developed in Python (using PyTorch library) to generate weights.

- **Input Representation:** The system is designed to process complex-valued signals common in digital communications (QPSK). The input vector x_t consists of two features representing the In-Phase (I) and Quadrature (Q) components of the received symbol. The bits are represented as (I,Q) symbol format, where bits 00 are represented by (1,1), bits 01 by (-1,1), bits 10 by (-1,-1) and bits 11 by (1,-1).
- **Network Topology:** The core GRU layer consists of configurable hidden units (default parameter $N_{\text{hidden}} = 3$) to capture the channel memory. A linear layer maps the final hidden state to the estimated symbol.
- **Optimization Strategy:**
 - **Python Golden Model:** Utilizes the Adam optimizer, which uses variable learning rate for rapid convergence during software simulation benchmarks.
 - **FPGA Hardware:** While adaptive optimizers like Adam improve convergence speed, they require complex hardware logic for calculating running averages and square roots. To minimize resource utilization on the FPGA, a fixed learning rate (η) was chosen.

Before implementing the architecture in Verilog, the network hyperparameters were frozen to balance learning capability with FPGA resource constraints. The system parameters utilized for the experimental validation are summarized in Table I. QPSK was chosen as it provides twice the number of bits per symbol than BPSK, widely used in communication systems and provides complex representation of signals. The hidden size of three was chosen as to be sufficient for learning the non-linearities but also be computationally light for on-chip training. A sequence of three was chosen to reduce the number of unfolded GRU

steps, to lower the hardware memory but still be able to learn the temporal dependencies.

TABLE I
HYPERPARAMETERS

Parameter	Value / Description
Modulation	QPSK (Quadrature Phase Shift Keying)
Input Features	2 (In-Phase (I) and Quadrature (Q))
Hidden Units	3 (Configurable)
Output	2 (In-Phase (I) and Quadrature (Q))
Sequence Length	3
Precision	IEEE 754 Single Precision (32-bit)
Loss Function	Mean Squared Error (MSE)

B. Hardware Inference Implementation

To manage the complexity of the GRU-BPTT algorithm and make it modular, the hardware is designed using a hierarchical approach. The system is built from fundamental arithmetic units (FPU) up to the top-level training controller and inference module.

- 1) **Dot Product Unit (dot_product)** The fundamental operation in any neural network is the weighted sum ($\sum w_i x_i$). This module implements a serialized vector-vector multiplication. It iterates through the input vector elements, utilizing the shared single floating-point multiplier and adder to compute the sum-of-products. By serializing this operation, we avoid instantiating large parallel adders, significantly saving FPGA Logic Elements (LEs).
- 2) **Activation Function Units (SigMoid, tanh)** These modules provide the non-linear capabilities of the network. Since calculating exponentials (e^{-x}) is costly in hardware, these units utilize Look-Up Tables (LUTs). The 32-bit floating-point input is used to address a ROM containing pre-computed values.
- 3) **Gate Calculation Unit (gate_cal)** This module abstracts the math for a single GRU gate. It instantiates two dot_product units to compute the input contribution (W_x) and the hidden state contribution (U_h) in parallel. It then adds the bias and passes the result through the appropriate activation function unit (Sigmoid or Tanh) based on a flag.
- 4) **GRU Cell (GRU_Cell)** This module implements the logic for a single time step. It operates as a sequencer that triggers the gate_cal unit three times in sequence:
 - **Step 1:** Compute Reset gate (r_t).
 - **Step 2:** Compute Update gate (z_t).
 - **Step 3:** Compute Candidate state (n_t).**Final Hidden State:** It performs the linear computation to produce the new hidden state. Crucially, this module exposes all internal gate values (r_t, z_t, n_t) to the top level for storing.
- 5) **GRU Layer (GRU_Layer_BPTT)** This module manages the temporal sequence. It contains the “State Register” that stores (h_{t-1}). It accepts the full input sequence array and iteratively feeds one time-step at a time into the GRU_Cell.

- 6) **Linear Output Layer (linear_layer)** After the GRU Layer processes the full sequence, the final hidden state (h_t) is passed to this module. It performs output computation to generate the final prediction (the estimated QPSK symbol).

C. Hardware Training Implementation

The BPTT Training Controller, a complex Finite State Machine (FSM) that manages the complete forward and backward passes, for training the neural network.

- 1) **Inference Phase Management:** The controller begins in the inference start state. It triggers the GRU_Layer to process the input sequence. It monitors the “Done” signals and ensures that the variables required for gradients is correctly populated with the data of every gate activation.
- 2) **Backward Propagation Units:** Once the error is calculated (Loss), the controller switches to the training start state and activates the training-specific modules:

- **Backward Cell (GRU_Timestep_Backward):** This module implements the chain rule equations derived in the Theory section. It reads the r_t, z_t, n_t values from the stored registers (in reverse order, from T down to 1) and computes the error gradients with respect to the weights.
- **Weight Updater (weight_updater):** Using the gradients calculated above, this module applies the Gradient Descent (GD) weight update. It utilizes the shared floating-point subtractor to modify the weight registers in-place.

V. BLOCK DIAGRAM

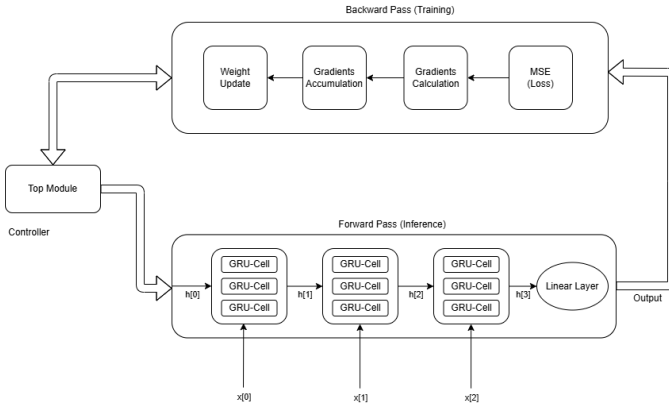


Fig. 4. Block diagram of Verilog modules.

The block diagram shown in Fig. 4 illustrates the overall architecture which was used to implement both the inference and training modules, which focused on scalability and clarity, to correct debugging and support different modulation techniques, with only minor changes. The top module stores all the weights and biases and has a switch for inference and training modes. The GRU_Cell is the core of the design, calculating all the gates and the final hidden state for each timestep. “Increasing the

number of hidden units instantiates additional GRU Cells in parallel without increasing the overall latency. Once GRU_Cells calculates the whole sequence, the output layer calculates the predicted symbol.

The BPTT modules calculate the loss, and the gradients and finally update the weights. The top module supervises the whole system.

VI. FLOW CHART

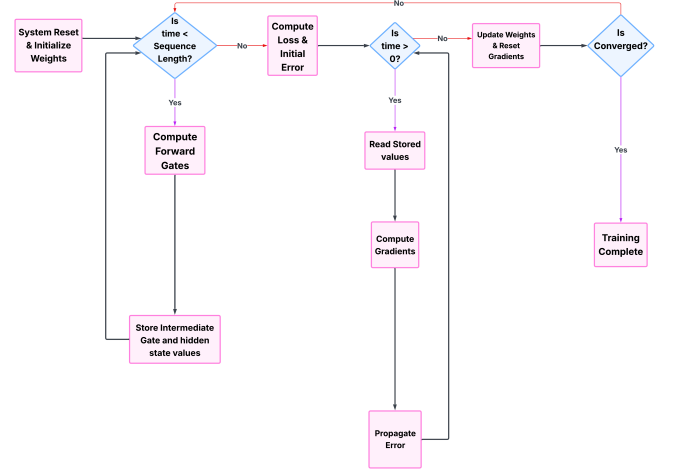


Fig. 5. Flowchart of the whole training algorithm.

The flowchart shown in Fig. 5, outlines the whole training algorithm. Firstly, the weights and biases are initialised, they can be random weights as to break symmetry at the start for faster convergence. Then the system enters the forward pass, where GRU_Cells compute the gates and hidden states and they are also stored for calculating the gradients. The output layer produces the predicted output and then the loss is calculated, if the loss is not converged, the gradients are calculated and the accumulated across total timesteps. Finally the weights are updated and then the loop continues until the loss is converged, or until a fixed number of epochs is reached.

VII. MATERIALS AND SOFTWARES USED

The hardware implementation and verification were conducted using the following platform specifications:

- **FPGA Development Board:** Terasic DE-10 Lite
- **Development Environment:** Intel Quartus Prime Lite Edition 20.1, Xilinx Vivado 2020.2
- **Simulation Tool:** ModelSim, Xilinx Vivado 2020.2
- **Synthesis Language:** Verilog HDL

VIII. RESULTS AND DISCUSSION

As observed in the Fig. 7, the MSE converged around 25 epochs and the weights converged around 175 epochs in Python as seen from Fig. 8 from weights L2 norm, which calculated mean squared values of all weights. The inference was run on the Python trained weights and it shows very good accuracy (MSE ;

0.001%) and correctly predicted all transmitted constellations as seen in Fig. 6. This proves that the model architecture is correct.

The RTL simulation showed decreasing trend in MSE for 5 samples in Fig. 9, decreasing from mse of 0.8931 to 0.0089 in 50 epochs, proving that all the modules are mathematically correct and working as expected.

The convergence can be made faster by implementing optimizers such as Adam, which keeps track of the first moment (moving average of the gradients) and the second moment (moving average of the squared gradients) of the gradients and adaptively changes the learning rate. This would again pose a hardware resource constraint but can be used if faster convergence is a priority and to reduce the oscillations and stuck gradients problem for large dataset.

The inference ran on DE 10 Lite board from Python exported weights which showed mse of 0.001373 (99.86% accuracy) and correctly predicted all the QPSK symbols as shown in Fig. 10, validating the correctness of the implemented architecture.

The MSE vs epoch as shown in Fig. 11 for learning rate = 0.001 and 10 sample size, shows the loss decreasing from 0.96 to 0.0187 in 200 epochs. The noise in the error plot is due to the LUT's precision and the non-adaptive learning rate. To improve the convergence, adaptive learning rate optimisers need to be employed to reduce oscillations and overshoot in convergence.

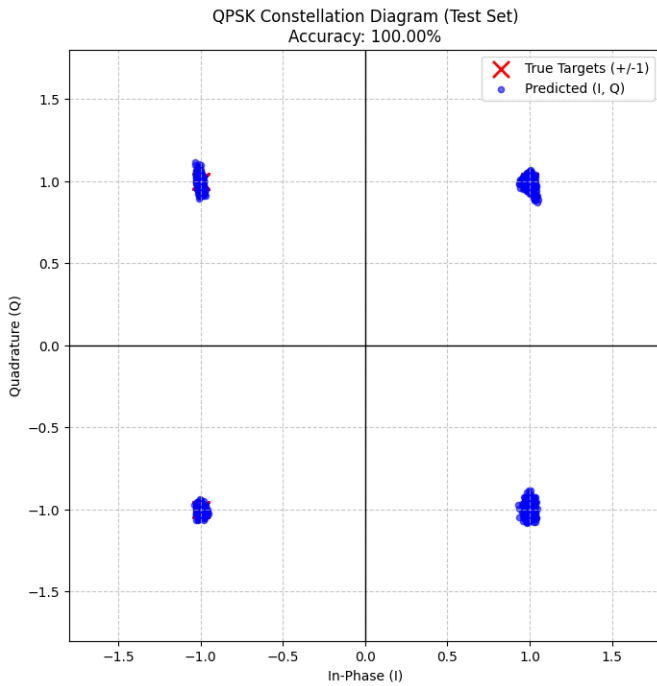


Fig. 6. QPSK Constellation diagram from the trained Python Model.

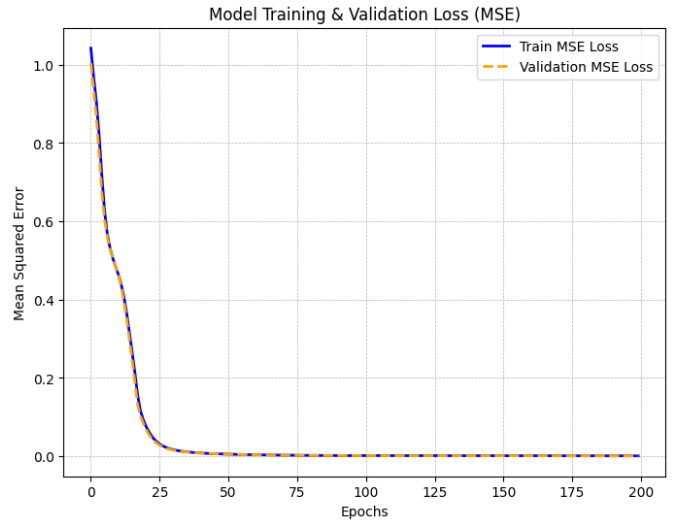


Fig. 7. MSE Vs epoch from the Python Model.

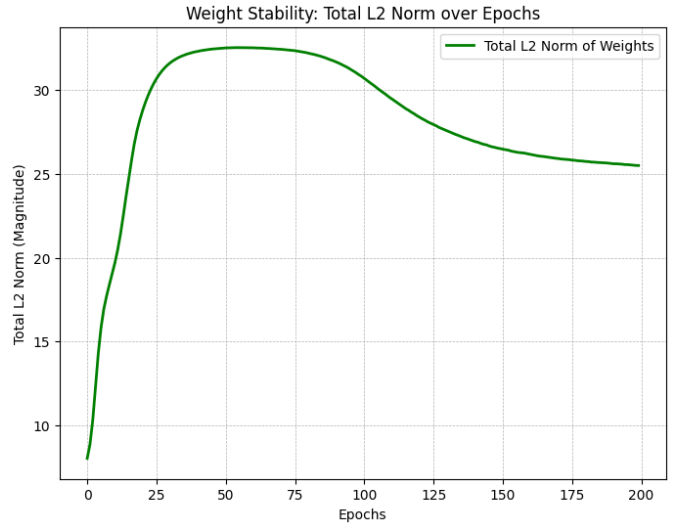


Fig. 8. Weights L2 norm Vs epoch from the Python Model.

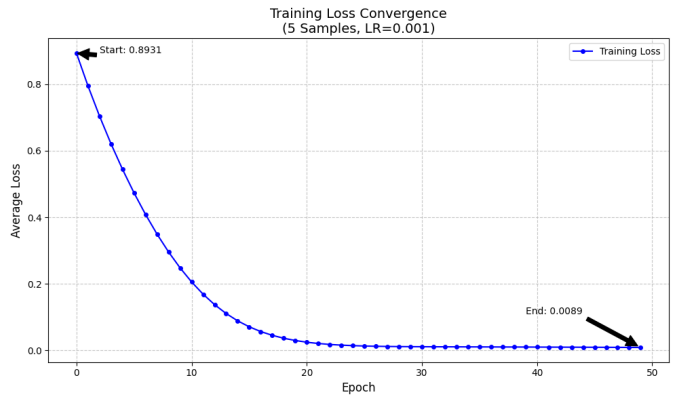


Fig. 9. MSE vs epoch from RTL Simulation.

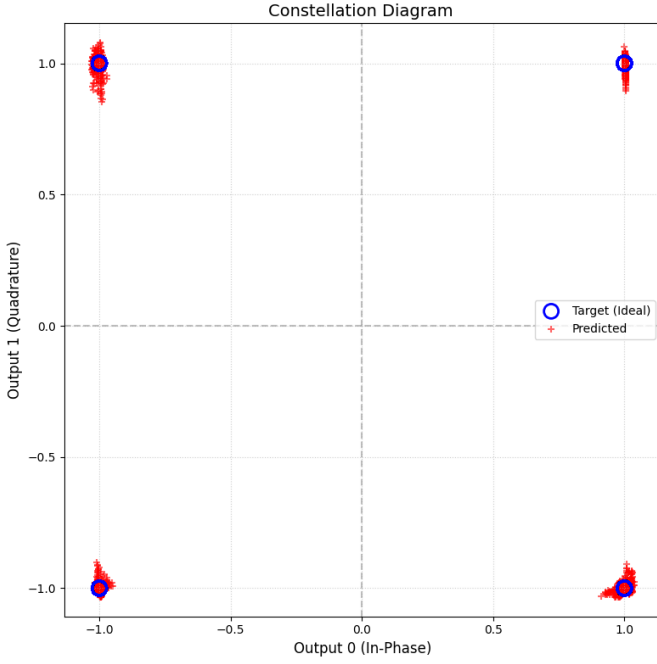


Fig. 10. Constellation Diagram for Python Exported weights on DE 10 Lite.

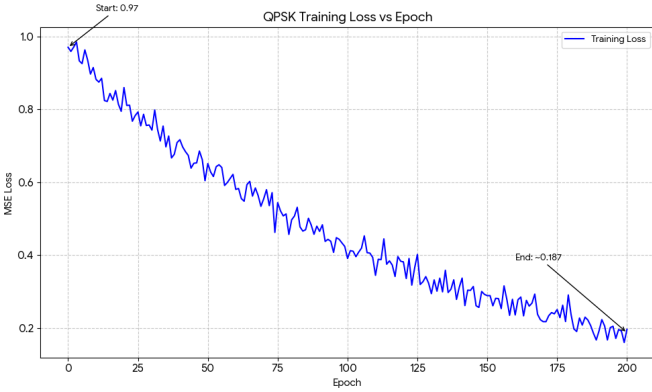


Fig. 11. MSE vs epoch plot for 10 samples and $\text{lr}=0.001$ on DE-10 Lite.

IX. CONCLUSION

This project successfully implemented a hardware accelerator for on-chip training of gated recurrent units (GRU), addressing the limitation of traditional “inference-only” FPGA designs. By translating the complex Backpropagation Through Time (BPTT) algorithm into a synthesizable Verilog architecture, the design enables the equalizer to calculate gradients and update weights dynamically without externally importing the weights.

X. FUTURE WORKS

To further enhance the performance and applicability of this on-chip trainer, the following improvements are proposed:

- 1) **Adaptive Optimizers:** Implementing Adam or RMSProp in hardware to accelerate convergence speed for fast changing channels.

- 2) **Precision Optimization:** Migrating from 32-bit Single Precision to emerging technology like Brain Float 16 (BF16) to reduce memory usage and logic footprint without sacrificing training stability [4].

These are just some of the optimizations, which can improve the model and make it more efficient and help the training have smoother and faster convergence. There can also be architectural changes proposed like going for LSTMs albeit with more complex architecture but captures long-range temporal dependencies.

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my project supervisor, Dr. Nandakumar Nambath, for his invaluable guidance, continuous encouragement, and technical insights throughout the duration of this project.

I am also deeply grateful to Mr. Akshay Gaonkar (PhD Scholar) for his mentorship and assistance with the FPGA implementation and verification process.

Finally, I thank the School of Electrical Sciences, IIT Goa, for providing the necessary laboratory infrastructure and FPGA development resources and necessary softwares required to complete this project successfully.

REFERENCES

- [1] A. Bhawar and A. Srivastava, “Multilayer perceptron equalizer using neural network accelerators,” IIT Goa, Goa, India, B.Tech. Project Rep., 2025.
- [2] P. J. Freire *et al.*, “Implementing neural network-based equalizers in a coherent optical transmission system using field-programmable gate arrays,” *J. Lightw. Technol.*, vol. 41, no. 12, pp. 3797–3812, Jun. 2023, doi: 10.1109/JLT.2023.3272011.
- [3] F. Ortega-Zamorano, J. M. Jerez, D. U. Muñoz, R. M. Luque-Baena, and L. Franco, “Efficient implementation of the backpropagation algorithm in FPGAs and microcontrollers,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 9, pp. 1840–1853, Sep. 2016, doi: 10.1109/TNNLS.2015.2460991.
- [4] Y. Umuroglu *et al.*, “FINN: A framework for fast and flexible quantized neural network inference,” *IEEE Trans. Comput.*, vol. 70, no. 4, pp. 631–644, Apr. 2021.
- [5] K. Cho *et al.*, “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Doha, Qatar, Oct. 2014, pp. 1724–1734, doi: 10.3115/v1/D14-1179.
- [6] S. Mandal *et al.*, “FPGA-based online learning using approximate hardware accelerators,” in *Proc. FPL*, 2021.